

DESIGN & ENGINEERING PROPOSAL

Admin UI Redesign: Components, UX and Wire-up

A plan to turn `apps/admin` from a shadcn dashboard starter into a working admin console. It covers a feature-based component structure, one app shell, the UX patterns every screen uses, removal of all mock data, and a full map of each UI action to the Convex function behind it. Where a function doesn't exist yet, its specification is included.

7

screens redesigned or added

~1,600

lines of mock and demo code removed

19

new Convex functions (+ 9 gain audit logging)

4

delivery phases

01	Current state audit	2	06	Screen designs	7
02	Mock data & dead code to remove	3	07	Functions & wire-up map	12
03	Target architecture	4	08	Backend (Convex) specification	13
04	Information architecture & shell	5	09	Delivery plan & acceptance	14
05	UX system & shared components	6			

SCOPE	STACK	DATE
apps/admin + packages/convex	Next.js 16 · React 19 · Convex · Clerk · shadcn/ui · Tailwind 4	29 September 2026

01 Current state audit

The admin app works for the basic CRUD paths, but it is still the shadow "dashboard-01" example with real pages bolted on. The problems fall into four groups.

A. Structure

FINDING	WHERE	IMPACT
Each page rebuilds the shell (<code>SidebarProvider</code> , <code>AppSidebar</code> , <code>SiteHeader</code> , <code>Toaster</code>)	<code>dashboard</code> , <code>documents</code> , <code>templates</code> , <code>templates/[id]</code>	Duplicated code; the shell re-mounts on every navigation, so the sidebar flickers and loses its state
<code>/user</code> page has no shell at all	<code>app/user/page.tsx</code>	Users page has no sidebar, so there's no way to navigate back without the browser
Page files of 400–890 lines that mix data, state, forms and dialogs	<code>templates/[id]</code> (890) , <code>documents</code> (470) , <code>templates</code> (400)	Hard to test or reuse; every change touches a large file
Two data-access styles: REST routes via <code>fetch</code> for documents and stats, reactive <code>useQuery</code> for users and templates	<code>api/documents</code> , <code>api/stats</code> , <code>section-card.tsx</code>	Documents list doesn't update live and re-fetches everything after every edit; stats load all rows just to count them
Mixed Button imports (<code>@repo/ui/button</code> vs <code>@repo/ui/components/ui/button</code>)	<code>user</code> , <code>documents</code>	Inconsistent styling and variants
Widespread <code>any</code> (<code>editingUser: any</code> , <code>form: Record<string, any></code> , <code>error: any</code>)	<code>user</code> , <code>templates/[id]</code> , <code>api routes</code>	No type safety on the most sensitive screens

B. UX

FINDING	WHERE	IMPACT
Header title is hardcoded as "Documents" on every page	<code>side-header.tsx</code>	Users can't tell where they are
No active state in the sidebar; links are <code><a href></code> , not <code>next/link</code>	<code>nav-main.tsx</code>	Full page reloads and no "you are here" cue
Search is client-side over the full list, with no filters, sorting or pagination	<code>documents</code> , <code>templates</code> , <code>user</code>	Won't scale, and the view can't be shared by URL
Users are shown twice (an Admins table plus an All Users table)	<code>user/page.tsx</code>	Wasted space; a role filter is the conventional pattern
Documents are edited as raw HTML in a <code><textarea></code>	<code>documents edit dialog</code>	Easy to corrupt content; there's no preview
Spinner-only loading states; no skeletons and no error or retry states	<code>all pages</code>	Layout shifts, and failures fail silently

C. Broken or misleading functions

FINDING	WHERE	IMPACT
The admin Delete button has no <code>onClick</code>	<code>user/page.tsx:170</code>	The button does nothing
Admins can edit user name and email, but <code>users.upsert</code> overwrites both from Clerk on the user's next sign-in	<code>user dialog</code> → <code>users.updateUser</code>	Edits are silently lost. Profile fields must be read-only because Clerk owns them
Nothing stops the last admin from being demoted by another admin, or the role from being set to any string (<code>role: v.string()</code>)	<code>users.updateUser</code>	Risk of locking every admin out
"Quick Create", "Inbox", Settings, Get Help, Search, Billing, Notifications and Account are placeholder controls	<code>nav-main</code> , <code>nav-secondary</code> , <code>nav-user</code>	Dead clicks erode trust in the tool
The "Convex Database: Connected" card only reflects whether <code>/api/stats</code> returned 200	<code>section-card.tsx</code>	A fake health signal

D. Missing admin capabilities

The app has no organizations view, no user detail, no audit trail of admin actions, no bulk actions, and no real analytics, even though the schema already holds `createdAt` on users, documents and organizations and `usageCount` on templates.

02 Mock data & dead code to remove

Everything below is either fake data or a placeholder for a feature that doesn't exist. Each item is removed or replaced by the real equivalent in §06–§08.

FILE / SYMBOL	ACTION	WHAT IT IS → REPLACEMENT
app/dashboard/data.json	Delete	614 lines of proposal-outline rows ("Cover page", "Eddie Lake"...). Replaced by the Recent activity feed and Newest documents table from real data.
components/data-table.tsx	Delete	741-line shadcn demo: drag-to-reorder rows, "Past Performance / Key Personnel" tabs, a <code>TableCellViewer</code> with a fake chart and a zod schema for mock rows. Replaced by a generic <code>components/data-table/*</code> kit (§05).
components/chart-area-interactive.tsx	Rewrite	Hardcoded <code>chartData</code> (desktop/mobile visitors, April–June 2024). Becomes <code>features/dashboard/growth-chart.tsx</code> , fed by <code>admin.stats.timeseries</code> .
components/section-card.tsx	Rewrite	Fetches <code>/api/stats</code> and fakes a "Connected" status. Becomes <code>KpiGrid</code> with real counts, 7-day deltas and sparklines from <code>admin.stats.overview</code> .
app-sidebar.tsx → <code>data.user</code>	Delete	"shadcn / m@example.com / avatars/shadcn.jpg" fallback user. <code>NavUser</code> reads Clerk only and shows a skeleton while it loads.
app-sidebar.tsx → <code>navClouds</code>	Delete	Capture / Proposal / Prompts items, which are never rendered.
app-sidebar.tsx → <code>documents</code> + <code>nav-document.tsx</code>	Delete	"Data Library / Reports / Word Assistant" links to <code>#</code> , plus Open/Share/Delete menus that do nothing.
app-sidebar.tsx → Lifecycle, Analytics, Projects	Delete	Links to <code>#</code> . Replaced by the real IA in §04.
"Acme Inc." + <code>IconInnerShadowTop</code>	Replace	Colres wordmark and an "Admin" badge.
nav-main.tsx Quick Create + Inbox	Replace	No-op buttons. Replaced by the <code>⌘K</code> command palette, which includes the "Create template" action.
nav-secondry.tsx (Settings / Get Help / Search)	Replace	Search moves to <code>⌘K</code> ; Settings becomes a real page; Get Help is removed. The misspelled file name is fixed at the same time.
nav-user.tsx Account / Billing / Notifications	Replace	Keep only "Manage account" (Clerk <code>openUserProfile()</code>), "Theme" and "Sign out". "CN" avatar fallback → real initials.
side-header.tsx GitHub link	Delete	Links to the shadcn repository. Replaced by breadcrumbs and the search trigger.
AdminGuard.tsx commented SVGs	Clean	Use <code>IconLock</code> / <code>IconLogout</code> from Tabler, which is already a dependency.
app/page.module.css	Delete	Not imported anywhere.
public/turborepo-*.svg, vercel.svg, next.svg, globe.svg, window.svg, file-text.svg	Delete	create-turbo starter assets that aren't referenced.
app/api/stats/route.ts	Delete	Replaced by a reactive Convex query. It currently loads every user, organization and document only to call <code>.length</code> .
app/api/documents/route.ts	Delete	Replaced by direct <code>useQuery</code> / <code>useMutation</code> . The client is already authenticated through <code>ConvexProviderWithClerk</code> , and Convex enforces <code>requireAdmin</code> itself.
lib/convexServer.ts	Keep	Kept for future server-side exports (CSV and so on); it no longer has callers after this change.
package.json: <code>@dnd-kit/*</code> (4), <code>zod</code>	Remove	Only used by the demo data-table. Add zod back if the form layer adopts it (§05).

03 Target architecture

A **feature-sliced** layout: routes stay thin, each domain owns its components and hooks, and shared primitives live in one place. The rule is that `app/**/page.tsx` only composes feature components and should never exceed ~40 lines.

new moved / rewritten removed

```
apps/admin/
├─ app/
│  ├─ layout.tsx                # html/body, ClerkProvider, ConvexClientProvider, ThemeProvider
│  ├─ (admin)/
│  │  ├─ layout.tsx            # route group: everything behind AdminGuard + AdminShell
│  │  │  └─ <AdminGuard><AdminShell>{children}</AdminShell></AdminGuard>
│  │  └─ loading.tsx · error.tsx # route-level skeleton + error boundary with retry
│  │     └─ dashboard/page.tsx
│  │        └─ documents/page.tsx · documents/[id]/page.tsx
│  │           └─ templates/page.tsx · templates/[id]/page.tsx
│  │              └─ users/page.tsx · users/[id]/page.tsx # renamed from /user (redirect in next.config)
│  │                 └─ organizations/page.tsx · organizations/[id]/page.tsx # new
│  │                    └─ activity/page.tsx # new: audit log
│  │                       └─ settings/page.tsx # new: catalog tools, theme, danger zone
│  └─ not-found.tsx
├─ api/documents api/stats dashboard/data.json page.module.css
├─ components/                  # app-wide, domain-agnostic
│  ├─ shell/                    admin-shell · app-sidebar · nav-main · nav-user · site-header · breadcrumbs · command-menu · theme-toggle
│  ├─ data-table/               data-table · toolbar · faceted-filter · column-header · pagination · row-actions · bulk-action-bar · view-options
│  ├─ feedback/                 empty-state · error-state · table-skeleton · confirm-dialog · status-badge · copy-button
│  └─ guard/                    admin-guard · access-denied · sign-in-screen · guard.module.css
├─ page-header · stat-card · user-avatar · relative-time · sparkline
├─ features/                    # one folder per domain
│  ├─ dashboard/                kpi-grid · growth-chart · recent-activity · top-templates · newest-documents · use-dashboard.ts
│  ├─ documents/                documents-table · columns · document-sheet · document-meta-form · content-preview · delete-document-dialog · use-documents.ts
│  ├─ templates/                templates-table · columns · create-template-dialog · delete-template-dialog · use-templates.ts
│  │   └─ editor/                template-editor · editor-header · general-tab · content-tab · sections-tab · fields-tab · latex-tab · license-tab · use-template-form
│  ├─ users/                    users-table · columns · user-sheet · role-select · change-role-dialog · delete-user-dialog · use-users.ts
│  ├─ organizations/            orgs-table · columns · org-sheet · members-list · use-organizations.ts
│  └─ activity/                 activity-feed · activity-item · activity-filters · use-activity.ts
├─ hooks/                       use-url-state · use-debounced-value · use-confirm · use-action (mutation+toast) · use-hotkey · use-current-admin
├─ lib/                         nav-config.ts · routes.ts · format.ts · slugify.ts · convex-error.ts · constants.ts · convexServer.ts
└─ middleware.ts
```

Principles

1 · Thin routes, fat features

A page reads URL params and renders one feature root. Business logic lives in `features/*/use-*.ts` hooks, which unit tests can mock.

2 · One data layer: Convex

All reads are `useQuery` or `usePaginatedQuery` (live), all writes are `useMutation` or `useAction`. No REST in between. Authorization lives in Convex (`requireAdmin`), and the route guard only improves UX.

3 · URL is the state

Search, filters, sort, page size and the open record (`?q=&status=&sort=&open=<id>`) sync to the URL through `use-url-state`, so views can be shared, bookmarked and navigated with the back button.

4 · Types from the schema

Use `Doc<'users'>`, `FunctionReturnType<typeof api.x.y>` and `Id<'documents'>`. No `any`, and no `id` as `never` casts. Form state is typed from the mutation's args.

5 · Every async state designed

Each data surface renders exactly one of *loading* → *skeleton*, *empty* → *EmptyState* with a CTA, *error* → *ErrorState* with a *retry*, or *data*.

6 · Destructive actions are explicit

A single `ConfirmDialog` shows the blast radius (for example "removes 42 chat messages, 8 comments") and requires typing the name for irreversible deletes. Every admin mutation writes an audit entry.

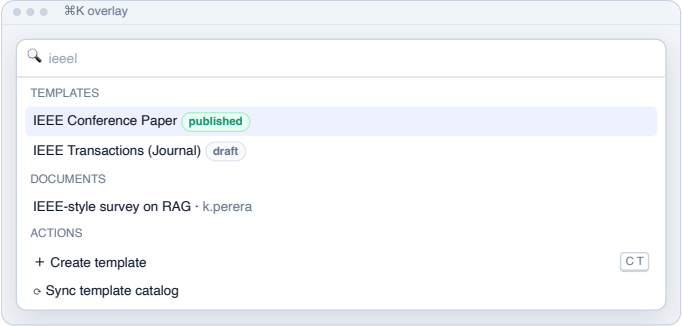
04 Information architecture & app shell

Navigation

GROUP	ITEM	ROUTE	BADGE
Overview	Dashboard	/dashboard	–
	Activity	/activity	–
Content	Documents	/documents	total
	Templates	/templates	drafts
People	Users	/users	total
	Organizations	/organizations	total
System	Settings	/settings	–

The nav is defined once in `lib/nav-config.ts` as `{group, title, href, icon, badgeKey}`. Sidebar, breadcrumbs and the command palette all read from it, so a new page is a one-line change. Badges come from `admin.stats.navCounts`, a single cheap query.

Command palette (⌘K)



Shell behaviour

- **AdminShell** mounts once in `(admin)/layout.tsx`, so the sidebar doesn't re-mount on navigation.
- Sidebar uses `collapsible="icon"` (instead of `offcanvas`) with tooltips. The collapsed state persists through the existing `sidebar_state` cookie. Toggle with `⌘B`.
- Active item uses `usePathname()` with `isActive` and matches nested routes (`/templates/abc` highlights **Templates**).
- Links use `next/link` with `prefetch`.
- **SiteHeader** shows breadcrumbs derived from the path and nav config (`Content › Templates › IEEE Conference`); the record name comes from a `BreadcrumbContext` that the detail page sets.
- Header right side: search trigger (`⌘K`), theme toggle, environment badge (`dev / prod` from the Convex URL).
- A single `<Toaster>` lives in the shell; the four per-page toasters are removed.
- Mobile: the sidebar becomes a sheet, tables collapse to card lists below `md`, and row actions move to a kebab menu.

Built on shadcn `Command` (`cmdk`). You'll need to add the `command` primitive to `@repo/ui`.

- **Navigate**: every nav item, plus `G` then `D/T/U/O` chords.
- **Search records**: debounced (200 ms) call to `admin.search.global({q})`, which returns up to 5 matches each from documents, templates, users and organizations via Convex search indexes.
- **Actions**: Create template, Sync catalog, Toggle theme, Sign out.
- Selecting a record navigates to its list with `?open=<id>`, which opens the detail sheet.

05 UX system & shared components

Built on the existing `@repo/ui` tokens (primary `#4f46e5`, radius `0.75rem`, Geist). No new colours are introduced; status colours map to semantic tokens so light and dark themes both work.

Reusable data-table kit: `components/data-table/`

One `TanStack-Table` wrapper used by Documents, Templates, Users, Organizations and Activity. Each feature supplies only `columns.tsx` and a filter config.

```
<DataTable
  columns={documentColumns} // ColumnDef<AdminDocument>[]
  query={usePaginatedQuery(api.admin.documents.list, filters, { initialNumItems: 25 })}
  toolbar={{ search: { placeholder: "Search title or slug..." },
    facets: [{ id: "status", options: STATUS_OPTIONS }, { id: "orgId", options: orgOptions }] }}
  bulkActions={[{ label: "Set draft", run: ids => bulkSetStatus({ ids, status: false }) },
    { label: "Delete", variant: "destructive", confirm: true, run: ids => bulkDelete({ ids }) }}]
  onClick={row => setOpen(row_id)} // opens the detail Sheet, writes ?open= to the URL
  empty={<EmptyState icon={IconFileText} title="No documents yet" />}
/>
```

Toolbar
Debounced search, faceted filter chips with counts, "Reset" button, and a column-visibility menu that persists in localStorage.

Rows
Checkbox selection, whole-row click opens the detail view, kebab menu for actions, and keyboard navigation (`↑↓`, `↵` to open, `⌘` to select).

Footer
"Showing 25 of 1,204", "Load more" / page-size switcher backed by Convex pagination cursors, and a sticky bulk-action bar when rows are selected.

Feedback & state components

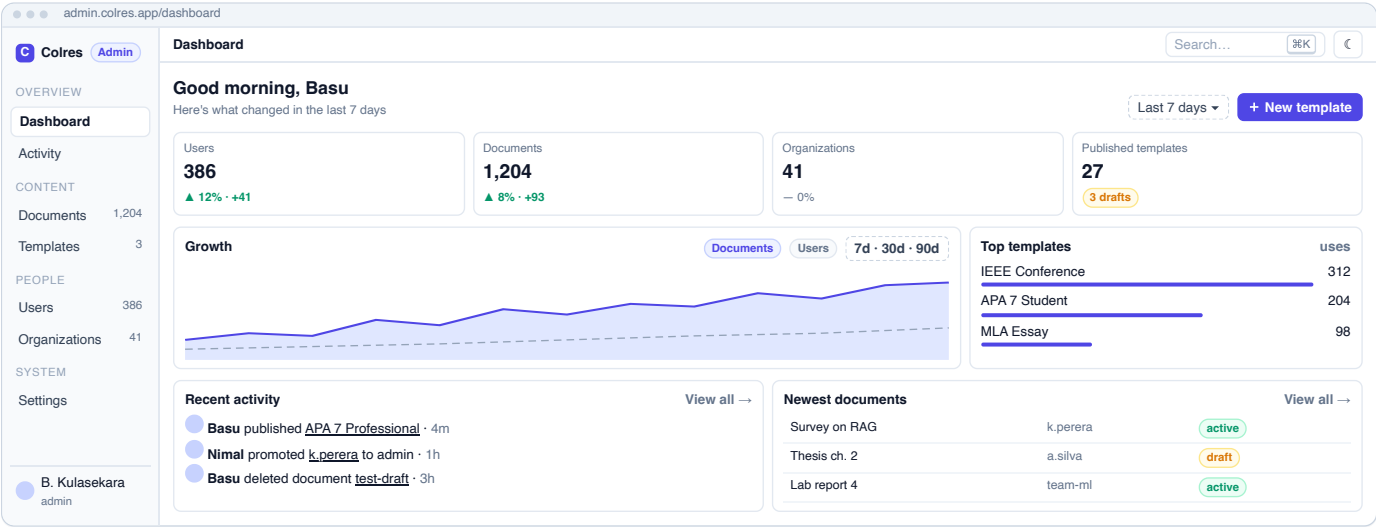
COMPONENT	BEHAVIOUR
TableSkeleton	Matches the column count and renders 8 shimmer rows. Used whenever <code>status === "LoadingFirstPage"</code> , so there's no layout shift.
EmptyState	Icon, title, one-line hint and an optional primary action. There are two variants: <i>no data</i> (with a CTA) and <i>no results</i> (with a "Clear filters" button).
ErrorState	Wraps a Convex error through <code>lib/convex-error.ts</code> (strips the "Uncaught Error:" prefix and maps "Unauthenticated" to a sign-in hint), with a Retry button that re-mounts the query.
ConfirmDialog	Imperative <code>await confirm({ title, description, impact, confirmText, requireTyping })</code> through <code>use-confirm</code> . It replaces the four hand-built delete dialogs.
use-action	<code>const [run, pending] = useAction(mutation, { success: "Saved", error: "Couldn't save" })</code> centralizes pending state, toasts and error mapping. It replaces ~20 duplicated <code>try/catch/toast</code> blocks.
StatusBadge	One map covers <code>published / draft / active / admin / user / featured</code> , with dot, label and semantic colour.
RelativeTime	"3 min ago", with a tooltip showing the absolute ISO timestamp. It replaces the mix of <code>toLocaleString()</code> and <code>toLocaleDateString()</code> .
CopyButton	Copies ids and slugs with an "Copied" tick. Used in sheets for debugging.

Interaction rules

- **Sheets for details, pages for heavy editing.** Documents, users and organizations open in a right-side `Sheet` so the list keeps its context. The template editor stays a full page.
- **Optimistic toggles.** Publishing or unpublishing a template and changing a document's status use `withOptimisticUpdate`, so the UI flips instantly and rolls back with a toast on error.
- **Unsaved-changes guard.** In the template editor, a sticky save bar appears when the form is dirty. The page blocks `beforeunload` and in-app navigation, and `⌘ S` saves.
- **Undo over confirm** for reversible actions (unpublish, role change back to user): a toast with an "Undo" button lasts 6 s.
- **Accessibility:** every icon button has an `aria-label` and tooltip, focus returns to the trigger when a dialog closes, contrast is AA, and nothing is conveyed by colour alone.
- **Dark mode** via `next-themes`. The `.dark` tokens already exist in `@repo/ui/globals.css`.

06 Screen designs

6.1 Dashboard: /dashboard



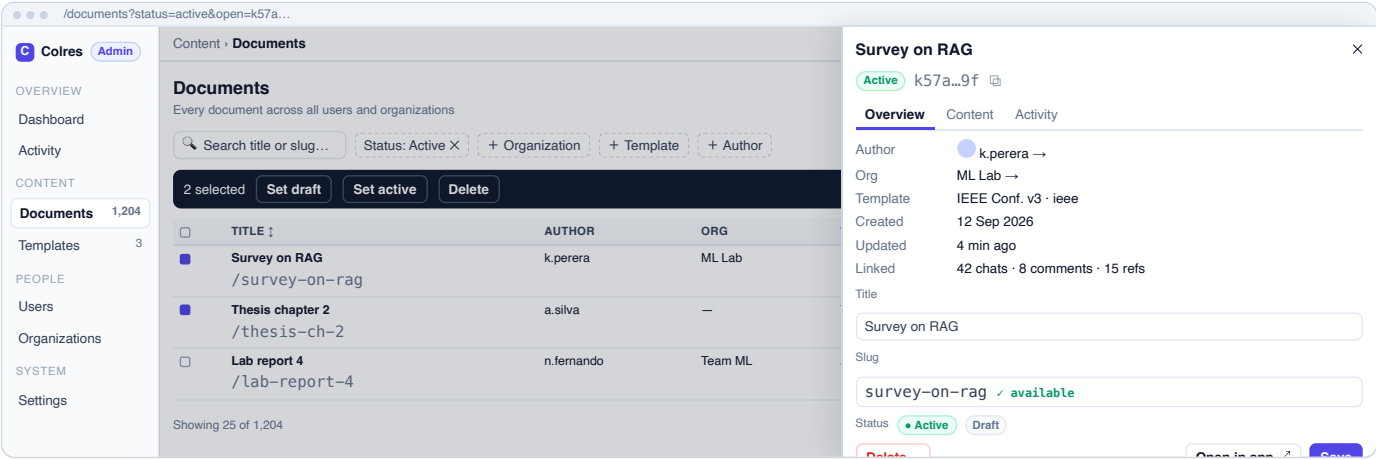
Components

- `KpiGrid` → 4 x `StatCard` showing value, delta versus the previous period and a sparkline. Clicking a card goes to its list with a matching filter (e.g. `Templates` → `?status=draft`).
- `GrowthChart` is a `Recharts` area chart with a series toggle and a range control. The range is stored in `?range=30d`.
- `TopTemplates` shows the top 5 by `usageCount` as bars.
- `RecentActivity` shows the last 8 `auditLog` rows.
- `NewestDocuments` shows the last 5 documents; clicking one opens the document sheet.

Data & states

- One query, `admin.stats.overview({ range })`, returns KPIs, deltas and sparklines. Another, `admin.stats.timeseries({ range })`, returns daily buckets.
- All widgets are reactive, so new sign-ups show up without a refresh.
- Each widget has its own skeleton and error boundary, so one failing widget doesn't blank the page.
- The greeting uses the Clerk first name and the local time of day.
- This removes `data.json`, `chart-area-interactive` mock data and `/api/stats`.

6.2 Documents: /documents + detail sheet



What changes

- Server-side **pagination and search** through a Convex `searchIndex("search_title")` on documents. This replaces loading every document with an N+1 author lookup.
- **Facets**: status, organization, template and author, each synced to the URL.
- **Detail sheet** instead of an edit modal:
 - *Overview* tab: metadata and a form for title, slug and status.
 - *Content* tab: a **read-only rendered preview** of the HTML, sanitized. Raw HTML editing is removed, because content is edited in the web app's TipTap editor, which the "Open in app" link reaches.
 - *Activity* tab: audit rows for this document.
- Slug field validates live (format and uniqueness) through `admin.documents.checkSlug`, debounced.

Wire-up

- List: `usePaginatedQuery(api.admin.documents.list, {q, status, orgId, templateId, authorId, sort})`
- Sheet: `useQuery(api.admin.documents.get, {id})`, which returns the author, org, template and linked counts
- Save: `documents.adminUpdateDocument` (existing), which now also writes an audit entry
- Status toggle: an optimistic update on the same mutation
- Delete: `ConfirmDialog` shows the linked counts and requires typing the slug, then calls `documents.adminDeleteDocument` (existing)
- Bulk: `admin.documents.bulkSetStatus` and `bulkDelete` (new), up to 100 ids per call
- Export: `admin.documents.exportCsv` action, which returns a file URL, or a client-side CSV of the current filter

6.3 Templates: list /templates

Content > Templates

Search...

⌘K

Templates

The gallery users pick from when creating a document

Sync catalog

+ New template

All 30

Published 27

Drafts 3

Featured 6

Search...

Category

/

TEMPLATE	CATEGORY	CLASS · ENGINE	CITATION	STATUS	USES	V
IEEE Conference Paper ★	Journal articles	IEEEtran · pdflatex	ieee	Published ↕	312 3	...
APA 7 Professional	Assignments	apa7 · xelatex	apa	Draft ↕	0 1	...

- **Status tabs** with counts replace scanning the whole table for drafts.
- An inline **publish switch** uses an optimistic update and an Undo toast.
- Row menu: Edit, Duplicate, Preview in gallery ↗, Feature/Unfeature, Delete.
- Delete is blocked when `usageCount > 0`; the dialog offers **Unpublish instead**.
- Grid view (thumbnails) versus table view is remembered per admin.
- **New template** opens a `CreateTemplateDialog` that asks for name, category and *start from* (Blank or Duplicate an existing one). It then routes to the editor. This replaces creating "Untitled template" silently.
- **Sync catalog** moves behind a confirm that explains what overwrites what, with an "overwrite existing" checkbox. It's also available in Settings.
- Wire-up: `templates.adminListTemplates`, `adminSetTemplateStatus`, `adminDuplicateTemplate`, `adminDeleteTemplate`, `seedTemplates.adminReseedCatalog` (all existing), plus `adminSetFeatured` (new, or through `adminUpdateTemplate`).

6.4 Template editor: /templates/[id]

Content > Templates > IEEE Conference Paper

IEEE Conference Paper

Published

v3

Used by 312 documents · edits apply to new documents only

General

Content

Sections 6

Fields 4

LaTeX

Licence

Abstract required

 150–250 words ▾

Introduction required

Title

Introduction

Key

introduction

Target / Max

600 / 900

Guidance

State the problem and the contribution...

+ Add section

Live preview

 as users see it

Abstract · Introduction · Related work · Method · Results · Conclusion

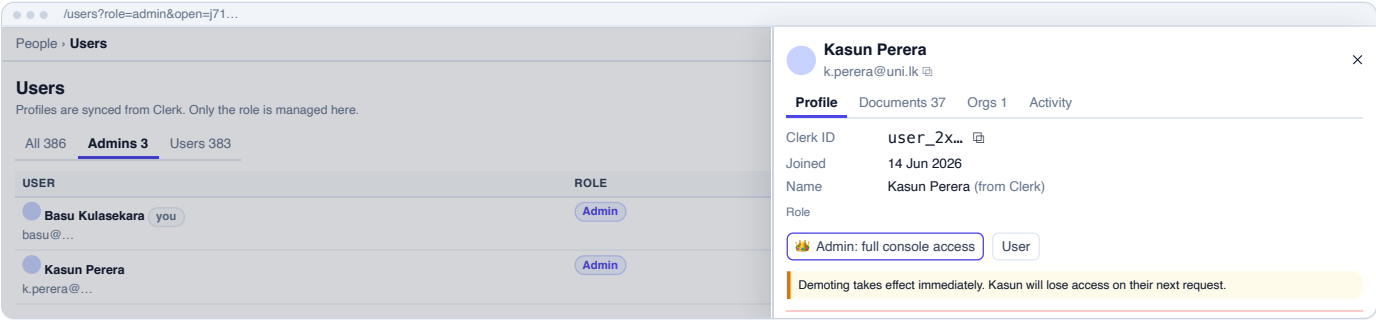
• Unsaved changes: Sections (2), Fields (1)

Discard

Save ⌘S

- The 890-line page is split into **6 tab components** plus `use-template-form.ts`, which uses `useReducer` for typed state, dirty tracking per tab and validation.
- Tab is stored in `?tab=`. Tab labels show counts and an error dot when a tab has invalid fields.
- **Sections and fields reorder by drag-and-drop** (`@dnd-kit/sortable`), which is why it stays as a dependency) with the keyboard as a fallback.
- Validation before save: unique keys, `target ≤ max`, and a slug format check. Errors show inline and the Save button is disabled with a reason.
- A sticky save bar shows *what* is dirty. `⌘S` saves, and the page blocks navigation while there are unsaved changes.
- A thumbnail uploader in General uses `adminGenerateThumbnailUploadUrl` (exists but unused today).
- Save calls `templates.adminUpdateTemplate`, which bumps `version`, writes an audit entry and shows a toast with the new version.

6.5 Users: /users + user sheet



Design decisions

- **One table with role tabs** replaces the duplicated Admins and All Users tables.
- **Name and email are read-only**, with a "Managed in Clerk" note. Today's edits are overwritten by `users.upsert` on the next sign-in, so allowing them is a bug.
- **Role change** uses a segmented control and a confirm that states the consequence. It's blocked for yourself ("You can't change your own role") and for the **last remaining admin**.
- **Delete user** is a real function (it was a no-op). The confirm shows owned documents and org memberships, offers *transfer documents to...* or *delete them*, and requires typing the email.
- The Documents tab lists this user's documents, and each row links to that document's sheet.

Wire-up

- List: `usePaginatedQuery(api.admin.users.list, {q, role, orgId})`, which adds `docCount` and `orgCount`
- Sheet: `useQuery(api.admin.users.get, {id})`
- Role: `api.admin.users.setRole({ userId, role })` (new), which replaces `users.updateUser` in the admin UI. `role` is `v.union("admin", "user")` and the last-admin guard runs server-side.
- Delete: `api.admin.users.remove`, an **action** that calls the Clerk Backend API (`clerkClient.users.deleteUser`) and then an internal mutation that reassigns or cascades documents and removes the row
- Avatar: `UserAvatar` falls back to initials. Clerk image hosts must be allowed in `next.config.js` `images.remotePatterns`; the current `<Image>` fails without it.

6.6 Organizations: /organizations new

• • • /organizations

Organizations

Team spaces synced from Clerk

🔍

Name or slug...

ORGANIZATION	MEMBERS	DOCS	UPDATED
ML Lab /ml-lab	12	88	2d
Team ML /team-ml	4	19	5d

- The list uses `api.admin.organizations.list` (paginated, with `memberCount` and `docCount`). It replaces the unused `getAllOrganizations` that only existed for the stats count.
- The sheet has an Overview tab (owner, created date, Clerk ID), a **Members** tab (joined from `members[]` and `admins[]` to user rows, each with an org-admin badge and a link to the user sheet) and a **Documents** tab (documents by `by_org_id`).
- This view is read-only because Clerk owns membership. It offers an "Open in Clerk dashboard ↗" link.

6.7 Activity (audit log): /activity new

• • • /activity?actor=...&type=template

Actor ▾

Entity: Template ×

Action ▾

Last 30 days ▾

TODAY

Basu published template [APA 7 Professional](#)

10:42

Basu updated template [IEEE Conference](#) v2 → v3 · sections, fields

10:15

YESTERDAY

Nimal changed role of [k.perera](#): user → admin

16:03

Basu deleted 3 documents

09:20

- This is new accountability for destructive and permission changes. Every admin mutation calls `logAudit(ctx, {...})`.
- The feed is grouped by day, and each entity name links to its sheet. Deleted entities show a snapshot of the name without a link.
- Wire-up: `usePaginatedQuery(api.admin.activity.list, {actorId, entityType, action, since})`.
- The same component is reused on the dashboard (limit 8) and in each sheet's Activity tab, filtered by `entityId`.

6.8 Settings: /settings new

SECTION	CONTENT	WIRE-UP
Appearance	Theme (system / light / dark) and density (comfortable / compact tables)	<code>next-themes</code> + <code>localStorage</code>
Template catalog	Sync the built-in catalog, with an "Overwrite existing templates" checkbox and a result summary (created / updated / skipped)	<code>seedTemplates.adminReseedCatalog</code>
Environment	Convex deployment URL, Clerk instance, app version and commit (read-only, for support)	<code>process.env.NEXT_PUBLIC_*</code>
Account	"Manage my account" opens the Clerk user-profile modal	<code>useClerk().openUserProfile()</code>

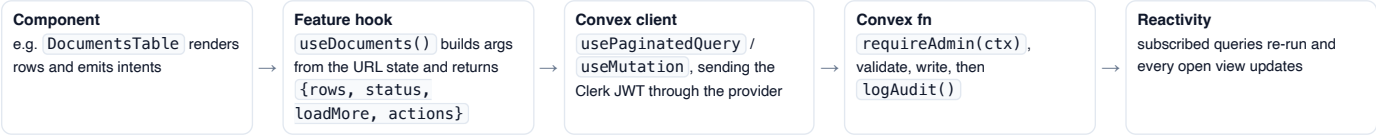
6.9 Access screens (AdminGuard)

The existing loading, sign-in and access-denied screens stay, but are split into `guard/sign-in-screen.tsx` and `guard/access-denied.tsx`, and they use the theme tokens instead of hardcoded hex. The access-denied screen gets "Switch account" (sign out, then back to sign-in) and "Go to Colres app ↗" buttons. The loading copy changes from "Verifying Credentials..." to a skeleton of the shell, which makes the load feel faster. A `useCurrentAdmin()` hook exposes the verified admin row to the shell (NavUser, "you" badges, self-guard) so no component queries it again.

07 Functions & wire-up map

Every interactive control and the function it calls. exists means the Convex function is used as-is (plus an audit call), change means a signature or behaviour change, and new is specified in \$08.

Data flow



SCREEN	UI ACTION	FUNCTION	STATUS
Shell	Sidebar badges	admin.stats.navCounts	new
	%K record search	admin.search.global({q})	new
	Current admin (avatar, "you", self-guards)	users.getByClerkId → useCurrentAdmin()	exists
	Profile sync on sign-in	users.upsert	exists
	Manage account / Sign out	clerk.openUserProfile() / signOut()	Clerk
Dashboard	KPI cards + deltas + sparklines	admin.stats.overview({range})	new
	Growth chart + range switch	admin.stats.timeseries({range})	new
	Top templates	admin.stats.topTemplates({limit:5})	new
	Recent activity	admin.activity.list({paginationOpts:{numItems:8}})	new
	Newest documents	admin.documents.list({sort:"createdAt", numItems:5})	new
Documents	List / search / filter / sort / paginate	admin.documents.list	new
	Open detail sheet	admin.documents.get({id})	new
	Slug availability	admin.documents.checkSlug({slug, excludeId})	new
	Save title / slug / status	documents.adminUpdateDocument	+audit
	Delete	documents.adminDeleteDocument	+audit
	Bulk set status	admin.documents.bulkSetStatus({ids, status})	new
	Bulk delete	admin.documents.bulkDelete({ids})	new
	Export CSV	client-side from the current filter (phase 1); admin.documents.exportCsv action (phase 4)	new
Templates	List + status tabs + counts	templates.adminListTemplates	exists
	Categories (filter, form select)	templates.adminListCategories	exists
	Create (dialog)	templates.adminCreateTemplate	+audit
	Publish switch (optimistic)	templates.adminSetTemplateStatus	+audit
	Feature / unfeature	templates.adminSetFeatured({id, featured})	new
	Duplicate	templates.adminDuplicateTemplate	+audit
	Delete (blocked if used)	templates.adminDeleteTemplate	+audit
	Editor load / save	templates.adminGetTemplate / adminUpdateTemplate	+audit
	Thumbnail upload	templates.adminGenerateThumbnailUploadUrl → POST → save thumbnailId	exists, unwired
	Sync catalog	seedTemplates.adminReseedCatalog({overwrite})	+audit
Users	List / search / role tabs	admin.users.list	new
	Open sheet (docs, orgs)	admin.users.get({id})	new
	Change role	admin.users.setRole({userId, role})	new
	Delete user	admin.users.remove({userId, documents:"transfer" "delete", transferTo?}) (action)	new
Orgs	List	admin.organizations.list	new
	Open sheet (members, docs)	admin.organizations.get({id})	new
Activity	Feed + filters	admin.activity.list	new

Retire after migration: `users.updateUser` and `users.editAdmin` (superseded by `setRole`; `editAdmin` has no callers), `organizations.getAllOrganizations` and `documents.getAllDocuments` (superseded by the paginated lists). All four are only called from `apps/admin` today (checked: no callers in `apps/web`).

08 Backend (Convex) specification

New admin functions live under `packages/convex/convex/admin/` (`stats.ts`, `documents.ts`, `users.ts`, `organizations.ts`, `activity.ts`, `search.ts`), so the admin surface is one folder that is easy to audit. Every function starts with `await requireAdmin(ctx)`.

Schema additions

```
// schema.ts
export const roleValidator = v.union(v.literal("admin"), v.literal("user"));

auditLog: defineTable({
  actorId: v.id("users"),
  action: v.string(), // "template.publish" | "user.role" | "document.delete" | ...
  entityType: v.union(v.literal("document"), v.literal("template"), v.literal("user"), v.literal("organization"), v.literal("catalog")),
  entityId: v.optional(v.string()),
  entityLabel: v.string(), // snapshot so deleted entities still read well
  meta: v.optional(v.any()), // { from: "user", to: "admin" }, { count: 3 }, ...
  createdAt: v.number(),
}).index("by_created", ["createdAt"])
  .index("by_entity", ["entityType", "entityId", "createdAt"])
  .index("by_actor", ["actorId", "createdAt"]);

// documents: add .index("by_status_updated", ["status", "updatedAt"]).index("by_created", ["createdAt"])
  .searchIndex("search_title", { searchField: "title", filterFields: ["status", "orgId", "author", "templateId"] })
// users: add .index("by_role", ["role"]).index("by_created", ["createdAt"])
  .searchIndex("search_name", { searchField: "name", filterFields: ["role"] })
// organizations: add .searchIndex("search_name", { searchField: "name" })
```

Key function signatures

```
// admin/stats.ts
overview = query({ args: { range: v.union("7d", "30d", "90d") } })
  → { users, documents, organizations, publishedTemplates, draftTemplates,
    deltas: { users, documents, organizations }, // current period vs previous, using by_created indexes
    sparklines: { users: number[], documents: number[] } }
timeseries = query({ args: { range } }) → Array<{ date: "2026-09-29", documents: number, users: number }>
topTemplates = query({ args: { limit: v.number() } }) → Array<{ _id, name, usageCount }>
navCounts = query({}) → { documents, users, organizations, draftTemplates }

// admin/documents.ts
list = query({ args: { paginationOpts, q: v.optional(v.string()), status: v.optional(v.boolean()),
  orgId, authorId, templateId, sort: v.union("updatedAt", "createdAt") } })
  // q ? withSearchIndex("search_title") : withIndex(...).order("desc"); join author/template per page only
get = query({ args: { id: v.id("documents") } }) → doc + author + org + template + { chats, comments, references }
checkSlug = query({ args: { slug, excludeId? } }) → { valid: boolean, available: boolean }
bulkSetStatus = mutation({ args: { ids: v.array(v.id("documents")), status: v.boolean() } }) // max 100
bulkDelete = mutation({ args: { ids: v.array(v.id("documents")) } }) // reuses cascadeDeleteDocument

// admin/users.ts
list = query({ args: { paginationOpts, q?, role?: roleValidator, orgId? } }) // + docCount, orgCount
get = query({ args: { id: v.id("users") } }) // + documents (recent 20), orgs
setRole = mutation({ args: { userId: v.id("users"), role: roleValidator } })
  // throws if self; throws if demoting the last admin (count by_role = "admin")
remove = action({ args: { userId, documents: v.union("transfer", "delete"), transferTo: v.optional(v.id("users")) } })
  // 1) runQuery requireAdmin + guards 2) Clerk: users.deleteUser(clerkId) 3) runMutation internal.admin.users._purge

// admin/activity.ts + lib/audit.ts
list = query({ args: { paginationOpts, actorId?, entityType?, entityId?, since? } })
export async function logAudit(ctx: MutationCtx, actor: Doc<"users">, e: Omit<AuditRow, "actorId"|"createdAt">)

// admin/search.ts
global = query({ args: { q: v.string() } }) → { documents: [...5], templates: [...5], users: [...5], organizations: [...5] }
```

Counting at scale. Convex has no `COUNT(*)`. Up to roughly 10k rows, counting over an index range is fine. Beyond that, add the `@convex-dev/aggregate` component (or a `counters` table maintained in the same mutations) so `overview` and `navCounts` stay $O(1)$. The API shape above doesn't change either way.

Clerk secret. `admin.users.remove` needs `CLERK_SECRET_KEY` set as a Convex environment variable (`npm run convex env set`). Without it, the action should fail with a clear message rather than deleting only the Convex row, because the user would otherwise be recreated on next sign-in by `upsert`.

09 Delivery plan & acceptance criteria

Four phases, each shippable on its own. Each phase ends green on `pnpm lint`, `pnpm check-types` and `pnpm build --filter admin`.

Phase 1 - Foundation & cleanup

no backend changes

- Create the `(admin)` route group, with `layout.tsx` mounting AdminGuard and AdminShell once. Remove the per-page shell and Toaster copies.
- Rename `/user` to `/users` with a permanent redirect. The page now sits inside the shell.
- Add `lib/nav-config.ts`; an active nav state using `next/link`; breadcrumbs; and a Colres brand block.
- Delete everything in `$02` (mock JSON, demo data-table, placeholder navs, starter assets, unused dependencies).
- Build `components/data-table/*`, `feedback/*`, `use-confirm`, `use-action`, `use-url-state`, `StatusBadge` and `RelativeTime`.
- Move the Documents page to `useQuery(api.documents.getAllDocuments)` with client-side filtering for now, then delete `/api/documents`.
- Users page: one table with role tabs, name and email read-only, and the dead Delete button removed until phase 3.
- Add the Clerk image domain to `next.config.js`; add `next-themes` and a theme toggle.

Phase 2 - Real dashboard & audit

schema: auditLog, by_created indexes

- Add the `auditLog` table and `lib/audit.ts`, and call it from every existing admin mutation.
- Add `admin/stats.ts` (overview, timeseries, topTemplates, navCounts). Build the KPI grid, growth chart, top templates, recent activity and newest documents. Delete `/api/stats`.
- Add an `/activity` page and put the Activity tab component in sheets.

Phase 3 - Scalable lists & people management

schema: search indexes, roleValidator

- Add `admin/documents.ts` (list, get, checkSlug, bulk*). Build the documents table with facets, the detail sheet, the content preview and bulk actions.
- Add `admin/users.ts` (list, get, setRole, remove) with the self and last-admin guards and Clerk deletion. Build the user sheet and the danger zone.
- Add `admin/organizations.ts` and the `/organizations` page and sheet.
- Retire `updateUser`, `editAdmin`, `getAllDocuments` and `getAllOrganizations` after checking `apps/web` for callers.

Phase 4 - Template editor & power features

- Split `templates/[id]` into six tabs and `use-template-form`, with validation, a sticky save bar, `⌘S` and a navigation guard.
- Add drag-reorder for sections and fields, the thumbnail uploader, a create-template dialog, feature toggling and grid view.
- Add the `⌘K` command palette with `admin.search.global`, keyboard chords, a `/settings` page and a CSV export action.

Acceptance criteria (definition of done)

AREA	CRITERION
No mock data	<code>grep -ri "Eddie Lake\ Acme\ shadcn\ m@example\ desktop:" apps/admin</code> returns nothing, and every nav link resolves to a real page.
Page size	No <code>page.tsx</code> is over 60 lines and no component is over 250 lines. There's no <code>any</code> in <code>apps/admin</code> , enforced with lint <code>no-explicit-any: error</code> .
Live data	Changing a document in the web app updates the admin list and the dashboard KPIs without a refresh.
States	Every list and widget has a verified skeleton, empty state, no-results state and error-with-retry state.
Safety	You can't demote yourself or the last admin (checked server-side). Every destructive action shows its impact and appears in <code>/activity</code> .
URL state	Reloading <code>/documents?q=rag&status=active&open=<id></code> restores the same filtered view with the sheet open.
Performance	First list page renders in under 500 ms on 10k documents, with a paginated query and no full-table <code>.collect()</code> in admin list paths.
Accessibility	The full flow is keyboard-operable, axe reports no serious violations, and dark mode reaches AA contrast.